

Practica - 19 Operaciones raster

Luis Gálvez

Tabla de contenidos

1 Proposito de la practica	1
1.1 Ejercicio único: Caracterización del piso térmico nival en el Parque Nacional Natural Los Nevados	1
1.1.1 Desarrollo	2
1.2 Preguntas	6
1.2.1 Desarrollo	6

1 Proposito de la practica

1.1 Ejercicio único: Caracterización del piso térmico nival en el Parque Nacional Natural Los Nevados

Usted debe construir un flujo de trabajo computacional en **Python o R** que cumpla con los siguientes requerimientos técnicos, utilizando los servicios de Google Earth Engine para la adquisición de datos en el área del Volcán Nevado del Ruiz (Cráter Arenas):

1. Adquisición y Estandarización:

- Conéctese a GEE y descargue el Modelo Digital de Elevación (MDE) de la misión **SRTM** (USGS/SRTMGL1_003) para un área de 15x15 km centrada en las coordenadas -75.319, 4.892.
- Reproyecte el ráster descargado (WGS84) al sistema oficial de Colombia **MAGNA-SIRGAS Origen Nacional (EPSG:9377)**.

2. Geoprocesamiento Local:

- **Recorte:** Realice un recorte (*Clip*) que aísle exclusivamente el edificio volcánico utilizando un Bounding Box definido por usted.
- **Topografía:** Genere un modelo de sombreado (*Hillshade*) con un azimut de 315° y una altitud solar de 45° a partir del MDE reproyectado.

3. Visualización:

- Despliegue el mapa resultante aplicando una rampa de color adecuada (ej. **terrain** para el MDE o **Greys** para el sombreado).

1.1.1 Desarrollo

```
# -----  
# 1. LIBRERÍAS  
  
import ee  
import geemap  
  
from osgeo import gdal  
import rasterio  
from rasterio.plot import show  
import matplotlib.pyplot as plt  
  
# -----  
# AUTENTICACIÓN E INICIALIZACIÓN GEE  
  
# Ejecutar solo la primera vez  
ee.Authenticate()  
  
# Inicializar Earth Engine  
ee.Initialize()  
  
# -----  
# 3. DEFINICIÓN DEL ÁREA DE ESTUDIO  
  
lon = -75.319  
lat = 4.892  
  
centro = ee.Geometry.Point([lon, lat])  
  
area_estudio = centro.buffer(7500).bounds()  
  
# Bounding Box del edificio volcánico  
volcan_bbox = ee.Geometry.Rectangle([  
    -75.38, 4.84,  
    -75.25, 4.96  
)  
  
# -----  
# 4. DESCARGA DEL MDE SRTM DESDE GEE  
  
# Modelo Digital de Elevación SRTM  
srtm = ee.Image("USGS/SRTMGL1_003")  
  
# Recorte inicial
```

```

dem = srtm.clip(area_estudio)

# Clip final al edificio volcánico
dem_volcan = dem.clip(volcan_bbox)

# -----
# 5. EXPORTACIÓN LOCAL DEL RASTER

import os

# Crear directorios
os.makedirs("data_heavy/raster", exist_ok=True)

ruta_mde_4326 = "data_heavy/raster/dem_nevado_ruiz_4326.tif"

# Exportar desde Earth Engine
geemap.ee_export_image(
    dem_volcan,
    filename=ruta_mde_4326,
    scale=30,
    region=volcan_bbox,
    file_per_band=False
)

print("DEM descargado localmente")
# -----
# 6. REPROYECCIÓN A MAGNA-SIRGAS ORIGEN NACIONAL (EPSG:9377)

ruta_mde_9377 = "data_heavy/raster/dem_nevado_ruiz_9377.tif"

# Reproyección
ds_warp = gdal.Warp(
    ruta_mde_9377,
    ruta_mde_4326,
    dstSRS="EPSG:9377"
)

# Cerrar archivo y liberar memoria
ds_warp = None

print("Raster reproyectado a EPSG:9377")

# -----
# 7. PRODUCTOS TOPOGRÁFICOS

```

```

ruta_pendiente = "data_heavy/raster/dem_nevado_ruiz_slope.tif"
ruta_orientacion = "data_heavy/raster/dem_nevado_ruiz_aspect.tif"
ruta_sombreado = "data_heavy/raster/dem_nevado_ruiz_hillshade.tif"

# -----
# Pendiente (Slope)

ds_slope = gdal.DEMProcessing(
    ruta_pendiente,
    ruta_mde_9377,
    "slope"
)

ds_slope = None

# -----
# Orientación (Aspect)

ds_aspect = gdal.DEMProcessing(
    ruta_orientacion,
    ruta_mde_9377,
    "aspect"
)

ds_aspect = None

# -----
# Hillshade // Azimut = 315° //Altitud solar = 45°

ds_hill = gdal.DEMProcessing(
    ruta_sombreado,
    ruta_mde_9377,
    "hillshade",
    azimuth=315,
    altitude=45
)

ds_hill = None

print("Productos topográficos generados")

# =====
# MAPA 1: MODELO DIGITAL DE ELEVACIÓN (MDE)

```

```

with rasterio.open(ruta_mde_9377) as mde_src:

    fig, ax = plt.subplots(figsize=(10, 10))

    # Mostrar DEM
    show(
        mde_src,
        ax=ax,
        cmap='terrain'
    )

    # Configuración cartográfica
    ax.set_title(
        "Modelo Digital de Elevación - Nevado del Ruiz",
        fontsize=14
    )

    ax.set_axis_off()

    plt.show()

# =====
# MAPA 2: HILLSHADE

with rasterio.open(ruta_sombreado) as hill_src:

    fig, ax = plt.subplots(figsize=(10, 10))

    # Mostrar Hillshade
    show(
        hill_src,
        ax=ax,
        cmap='Greys_r'
    )

    # Configuración cartográfica
    ax.set_title(
        "Hillshade - Nevado del Ruiz",
        fontsize=14
    )

    ax.set_axis_off()

    plt.show()

```

1.2 Preguntas

1. **Sobre el control de CRS:** ¿Por qué es un error metodológico crítico intentar realizar álgebra de mapas combinando una banda verde proyectada en WGS84 Geográfico y una banda NIR en MAGNA-SIRGAS Plano, aunque visualmente parezcan abarcar la misma región colombiana?
2. **Sobre la eficiencia de ejecución:** Explique técnicamente por qué el cálculo de índices espectrales usando matrices puras (álgebra vectorizada) finaliza en milisegundos, frente al riesgo de implementar algoritmos manuales que usen ciclos `for` que iteren fila por fila y columna por columna sobre toda la escena del Nevado del Ruiz.
3. **Sobre librerías y memoria (Lazy Evaluation):** En el contexto de Python (`rioxarray`) y R (`terra`), describa el concepto técnico que permite extraer matemáticamente recortes espaciales a partir de archivos ráster masivos sin cargar la totalidad del archivo en la memoria RAM (Gestión de punteros y buffers).

1.2.1 Desarrollo

1. En algebra de mapas, algunas operaciones requieren la correspondencia celda a celda entre las bandas a utilizar. En el caso propuesto una banda se encuentra proyectada y cuenta con unidades metricas, mientras que la otra tiene un src no proyectado usando unidades angulares. Esto hace que aunque visualmente parezcan representar la misma área, existan diferencias en el área representada por cada pixel, además de que complejiza la ejecución de la operación matematica.
2. Se basa en la optimización y el consumo de recursos de la maquina. Las librerias como NumPy o rioXarray están optimizadas utilizando codigo en C para realizar estas operaciones (lo que las hace más veloces), mientras que el uso de ciclos `for` para recorrer dataframes tan extensos sobrecarga la memoria de la maquina y la exige demasiado.
3. Rioxarray en Python y terra en R implementan mecanismos de evaluación diferida (lazy evaluation) y lectura parcial de archivos raster, lo que significa que al abrir un GeoTIFF masivo, el software no carga inmediatamente todos los datos en memoria RAM. En cambio, crea objetos ligeros que contienen únicamente metadatos, referencias al archivo y punteros hacia bloques específicos del raster almacenado en disco.